



Permis d'illuminer toute la ville

LeHack 117 — Mission Olago

Agent 117 (en service commandé)

LeHack 2026

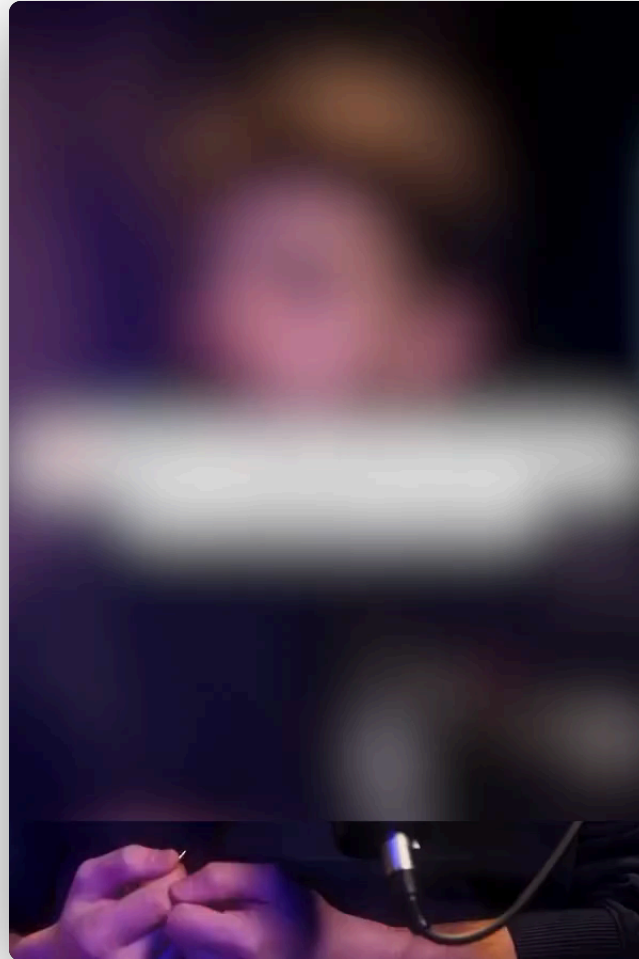
N°117

Quarkslab

**En résumé : cette présentation ne contiendra
aucune vulnérabilité**

N° 117

Tout commence par une vidéo



Le contexte (très important)



*Ce jour-là, j'avais une **réunion de deux heures**.*

Vous savez. Une *vraie* réunion.
Sur **l'avenir stratégique de la société**.

Le contexte (très important)



*Ce jour-là, j'avais une **réunion de deux heures**.*

Le moment **idéal** pour un petit challenge.
Dans le monde réel.



Hubert Squirrel de La Bath

Matricule 117

Le jour : embarqué & recherche de vulns à Quarkslab

Ce soir : agent secret, en mission



N°117

Quarkslab

Quelqu'un a dit que ce système était...

IM
PI
RA
TABLE

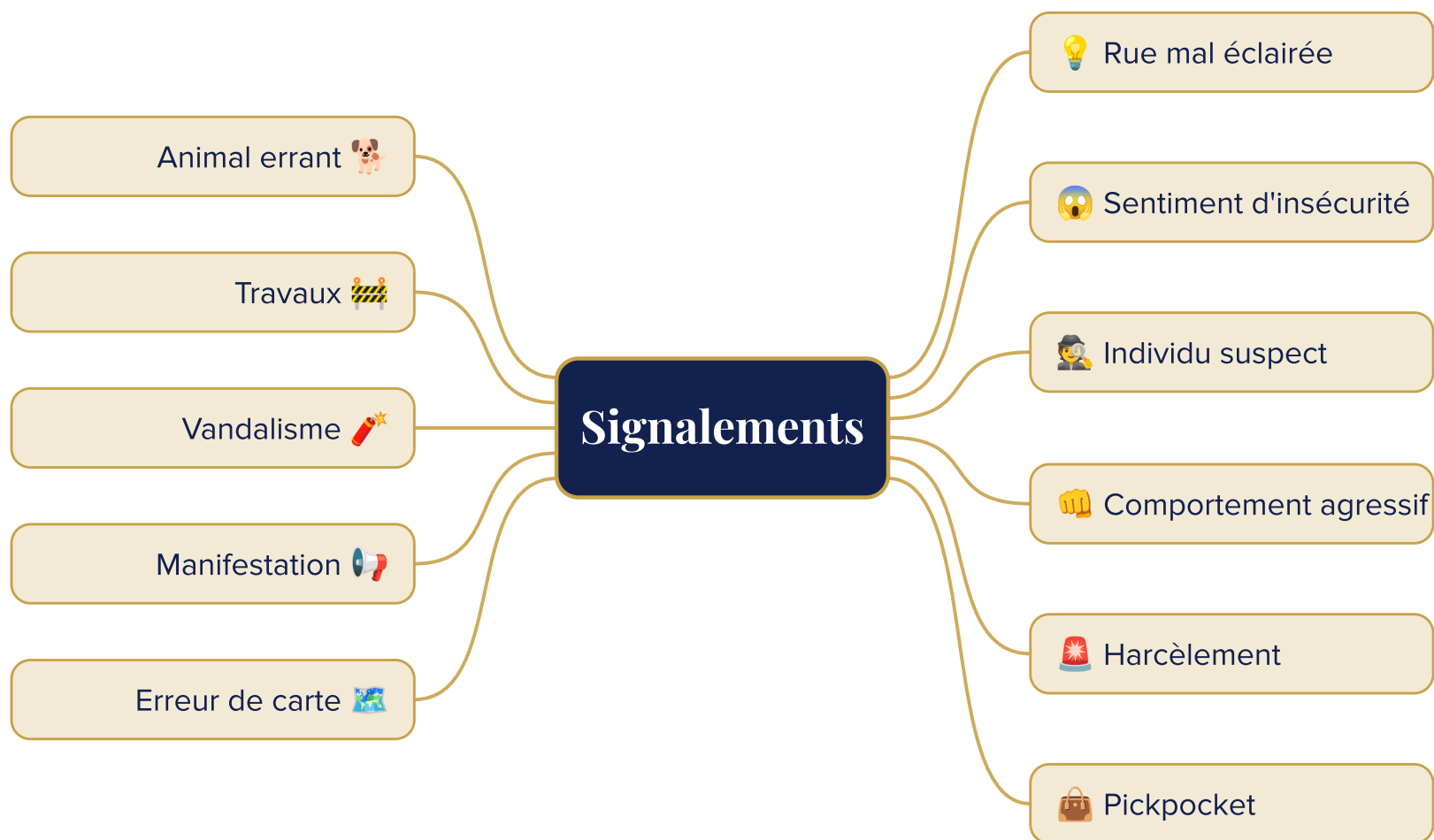
Étape 1 : on installe l'application



Télécharger sur le Play Store. Comme tout le monde. Aucun root, aucune bidouille, un citoyen modèle.

- ▶ 💡 **Allumer les lampadaires** de sa rue
- ▶ 📍 **Partager sa position** avec ses proches
- ▶ 🚨 **Signaler** un incident





Ouvrons le capot.

N° 117

Quarkslab

Avec **apkpatcher**, on dépaquette l'APK et on explore son contenu.

```
com.safeapplication.safe/ (App Bundle · React Native)
├─ base/
│   ├── AndroidManifest.xml
│   ├── classes*.dex           code Java / Kotlin
│   └─ assets/
│       └─ index.android.bundle ← bytecode Hermes ★
├─ config.arm64_v8a/lib/       libhermes.so – le moteur Hermes
├─ config.en/                  langues
└─ config.mdpi/                ressources
```

Du **React Native** compilé en **bytecode Hermes** et, à première vue, *pas grand-chose* côté protection.

On injecte Frida... et ✨



L'application **crash** dès que Frida s'accroche.

La protection : PairIP (RASP)



```
<activity android:name="com.pairip.licensecheck.LicenseActivity"  
    android:exported="false"/>  
  
<provider android:name="com.pairip.licensecheck.LicenseContentProvider"  
    android:exported="false"  
    android:authorities="com.safeapplication.safe.com.pairip.licensecheck.LicenseContentProvider"/>
```



Workshop apkpatcher
Cette nuit · 23h00 → 00h30
COMPLET OU PRESQUE

Le bypass : étape 1



```
# Patch APK + injection du gadget Frida + suppression des référence à pairIP  
apkpatcher -a com.safeapplication.safe.apk --download_frida --plugin remove_pairip.py
```



Le bypass : le résultat



```
python3 frida_runner.py    # chaîne de 7 scripts d'unpinning + bypass Conscript
```



Résultat : contrôle total



- ▶  L'APK patchée **se lance sans erreur**
- ▶  Tout le trafic **HTTPS apparaît en clair** dans mon reverse proxy (Burp)
- ▶  Le Bundle **Hermes décompilé** reste lisible bien que du Javascript c'est par défaut de l'obfuscation

Un travail rude. Quand on pense à toutes ces protections.

« Ne sera ***pas prise en compte***, car elle suppose un ***rootage*** du terminal utilisateur, ce qui sort du ***modèle de menace d'Olago***. »

— citation non contractuelle

Maintenant : allumons.

N° 117

Surprise : ce n'est pas Olago



En observant le trafic, l'API d'allumage n'est pas celle d'Olago.

C'est **jallume.fr** / en partenariat avec **Photon Group**.

3 étapes pour allumer une rue



1. Récupérer l'ID de la ville depuis une position

```
GET /app/Config/{lat}&{lon} → idVille # aucune authentification
```

3 étapes pour allumer une rue



2. Obtenir un token

```
await axios.post('https://auth.jallume.fr/Auth/JallumeToken', {  
  id:      userId,                // n'importe quelle chaîne (non validée côté serveur)  
  idVille: cityConfig.ville.id.toString(),  
});
```

3 étapes pour allumer une rue



3. Allumer

```
POST /App/LightRequest => { latitude: X, longitude: Y, token: JWT }
```

À aucun moment la position GPS n'est vérifiée côté serveur.

Le souci est intrinsèque



Pas de compte. Pas d'authentification. Pas de vérification GPS côté serveur.

Aucune protection technique ne peut empêcher ce scénario.

« Cette solution **ne repose sur aucun compte utilisateur**. [...] Ce mode de fonctionnement sans compte est un **comportement voulu et conscient** pour le parcours utilisateur et l'adoption. »

La réponse de Photon Group (2/3)



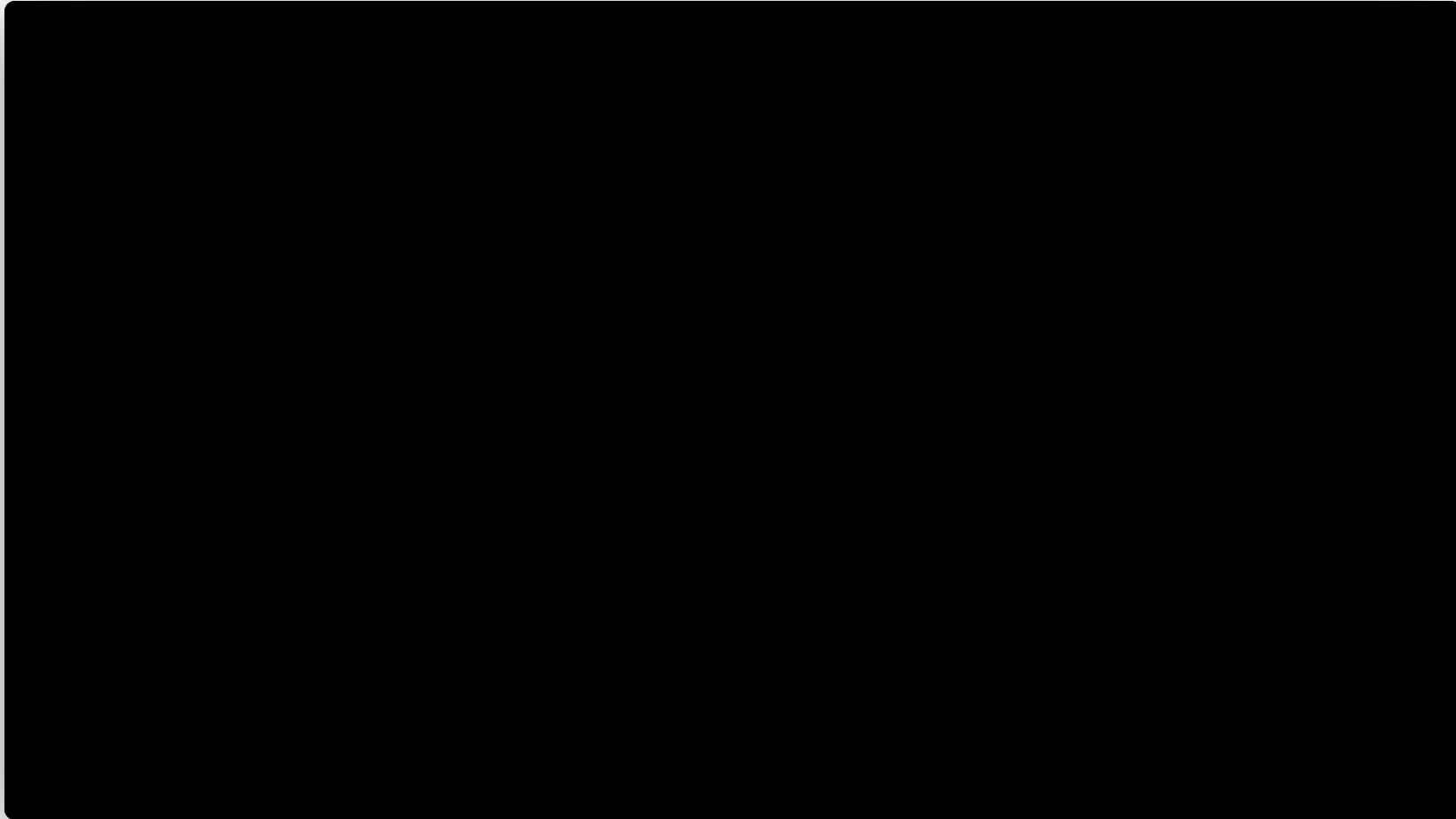
« L'ensemble des éléments permettant l'allumage [...] sont des **données provenant de l'appareil utilisateur** et sont donc **par définition falsifiables**. »

« Le token JWT n'est **pas destiné à authentifier** un utilisateur. »

La réponse de Photon Group (3/3)



« La vérification sur la distance de déplacement est **uniquement effectuée** côté **application client**, et **n'est pas vérifiée par le serveur**. »



Jour, nuit. À volonté.



On peut allumer une **ville entière, sans limite.**

Honnêteté : ce qu'on ne peut PAS faire



L'application n'offre **qu'un seul bouton** : « **allumer** » / jamais « éteindre ».

*Conséquence : **aucune extinction abusive n'est possible**, et le matériel reste donc protégé.*

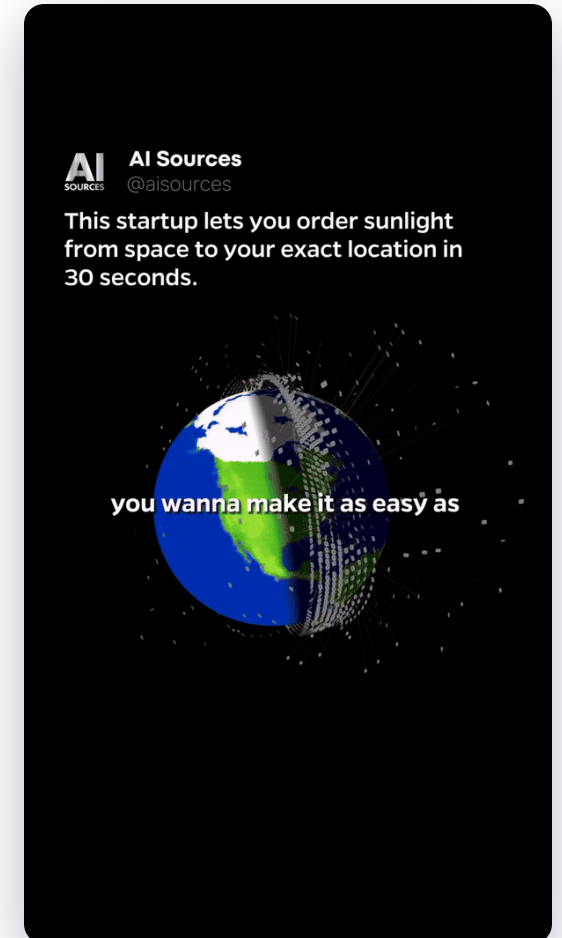
Quant à la durée d'allumage, elle est **fixe**...

Honnêteté : ...mais



- ▶ La durée d'allumage est **fixe**...
- ▶ ...mais **recupérable** via l'API.

*On peut donc relancer juste avant l'extinction.
Résultat : les lampadaires **ne s'éteignent jamais**.*



Mais... où sont les protections ?

N° 117

L'IA comportementale (la vraie)



```
if distance_parcourue > 50 and temps < seuil_secondes:  
    refuser()
```

La réunion n'est pas finie.

N° 117

Olago a ses propres serveurs (Firebase)



Trois usages :

- ▶  le **partage de position** en temps réel
- ▶  les **événements**
- ▶  les **incidents** signalés par les utilisateurs

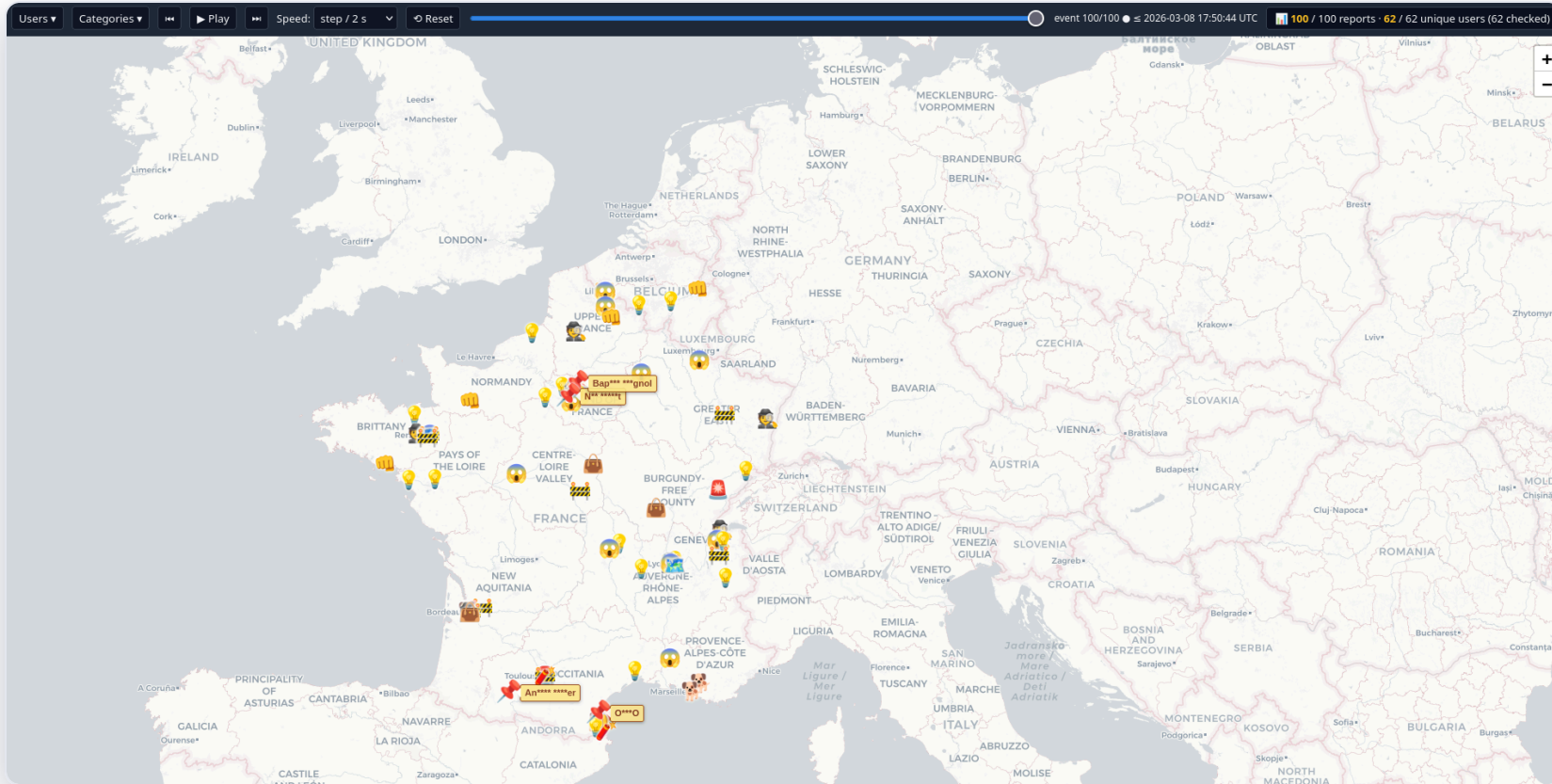
Le bucket parle trop



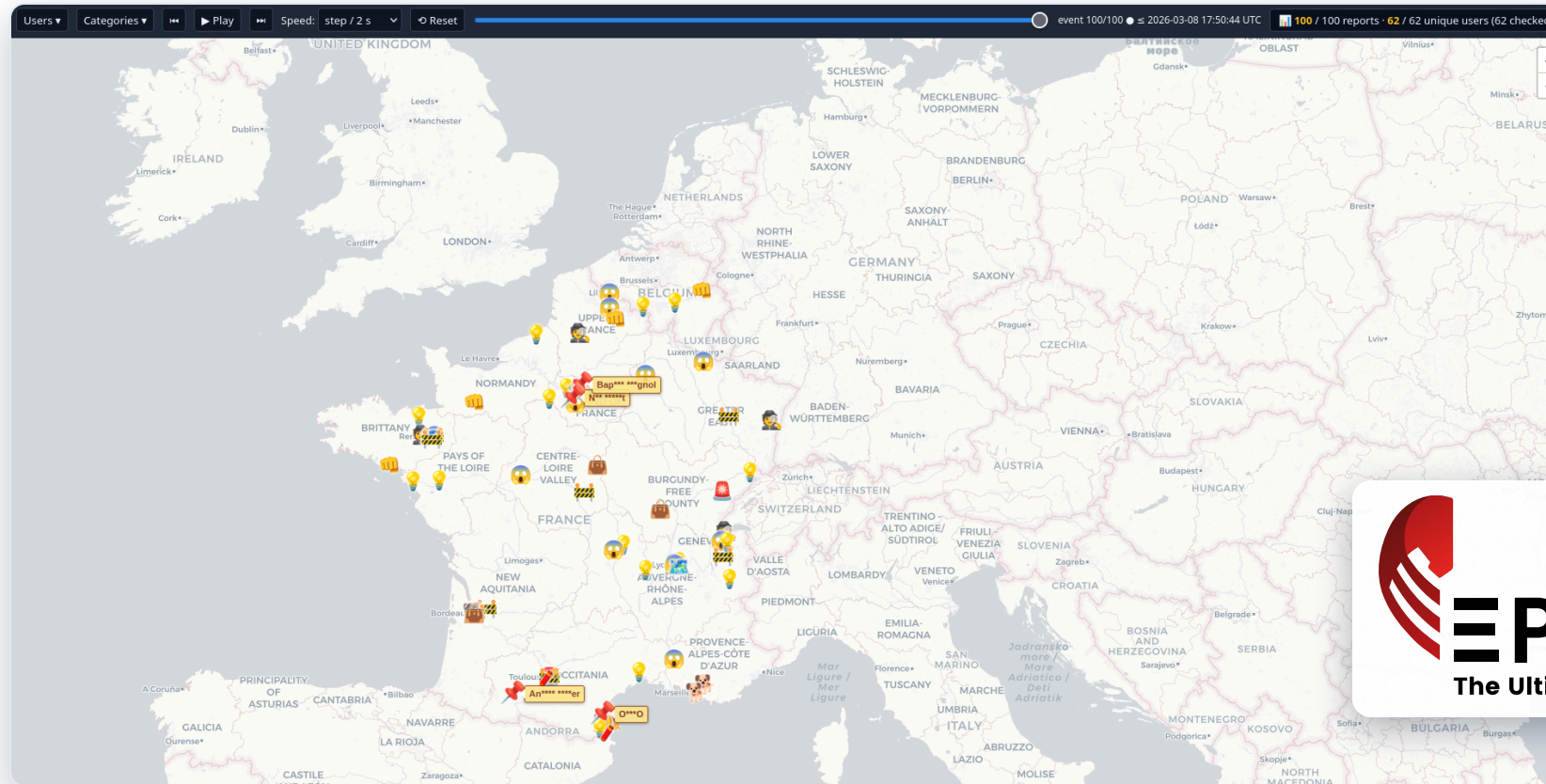
Les Cloud Functions filtrent... mais le **bucket Firebase** répond directement.

- ▶  **Tout** en une seule requête
- ▶  Des données **non exposées** par les fonctions :
 - ▶ les **codes d'accès** aux événements privés
 - ▶ l'**userId** de la personne qui a signalé (invisible dans l'UI)

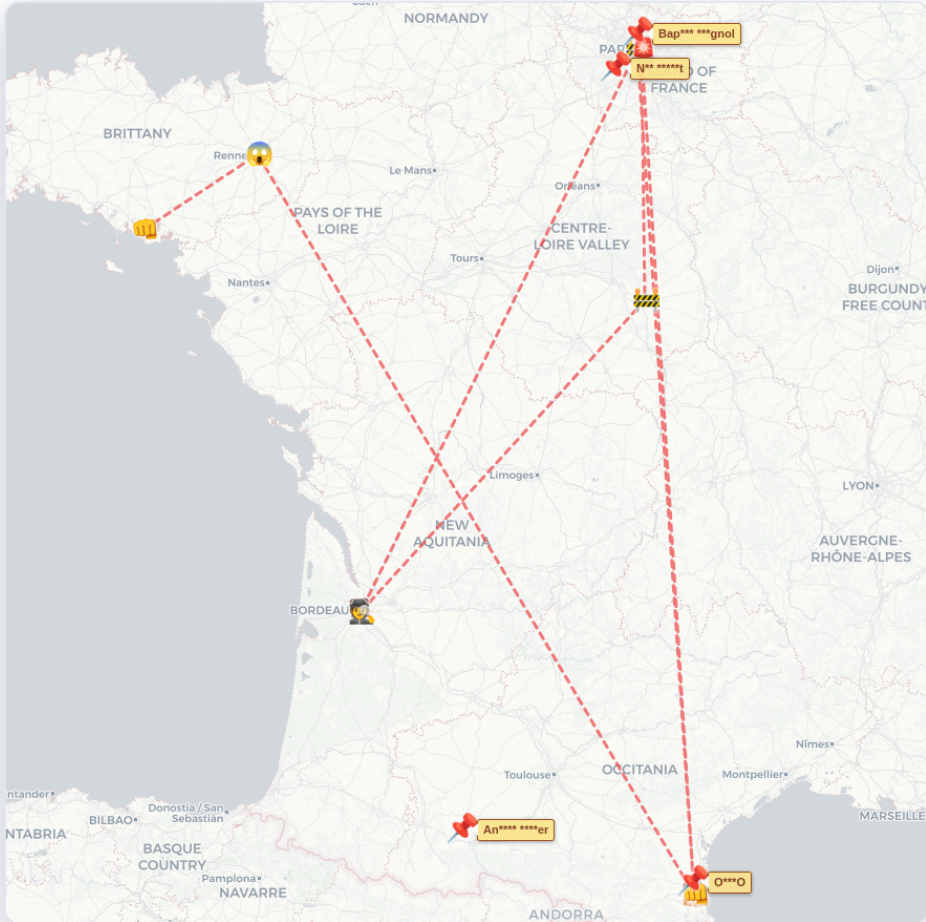
La carte de tout le monde



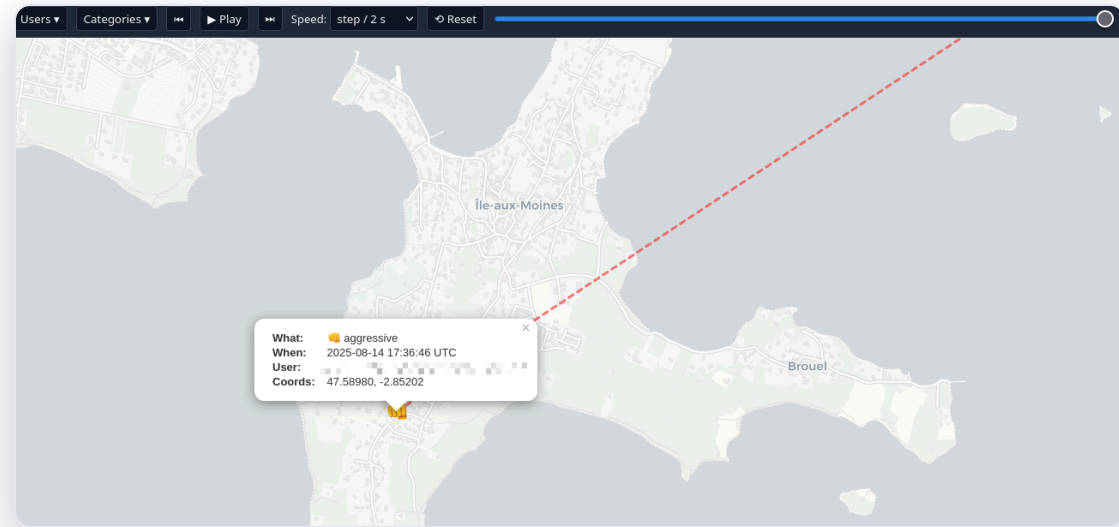
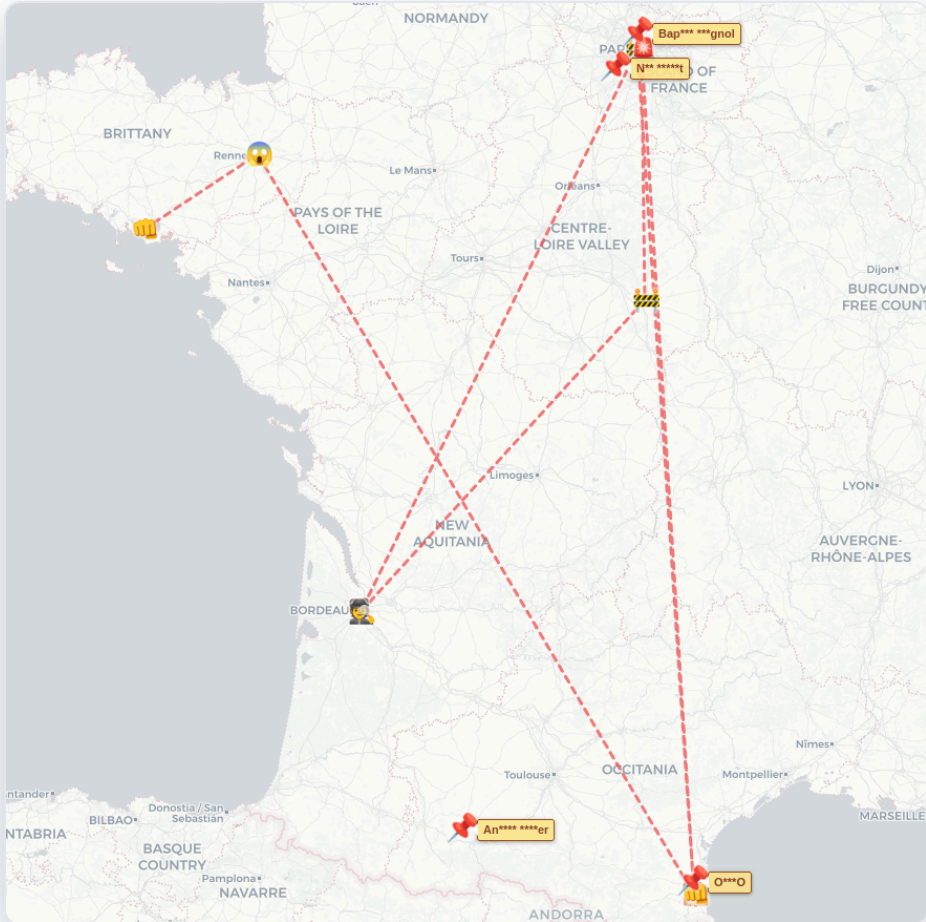
La carte de tout le monde



Un utilisateur... et son lieu de vacances



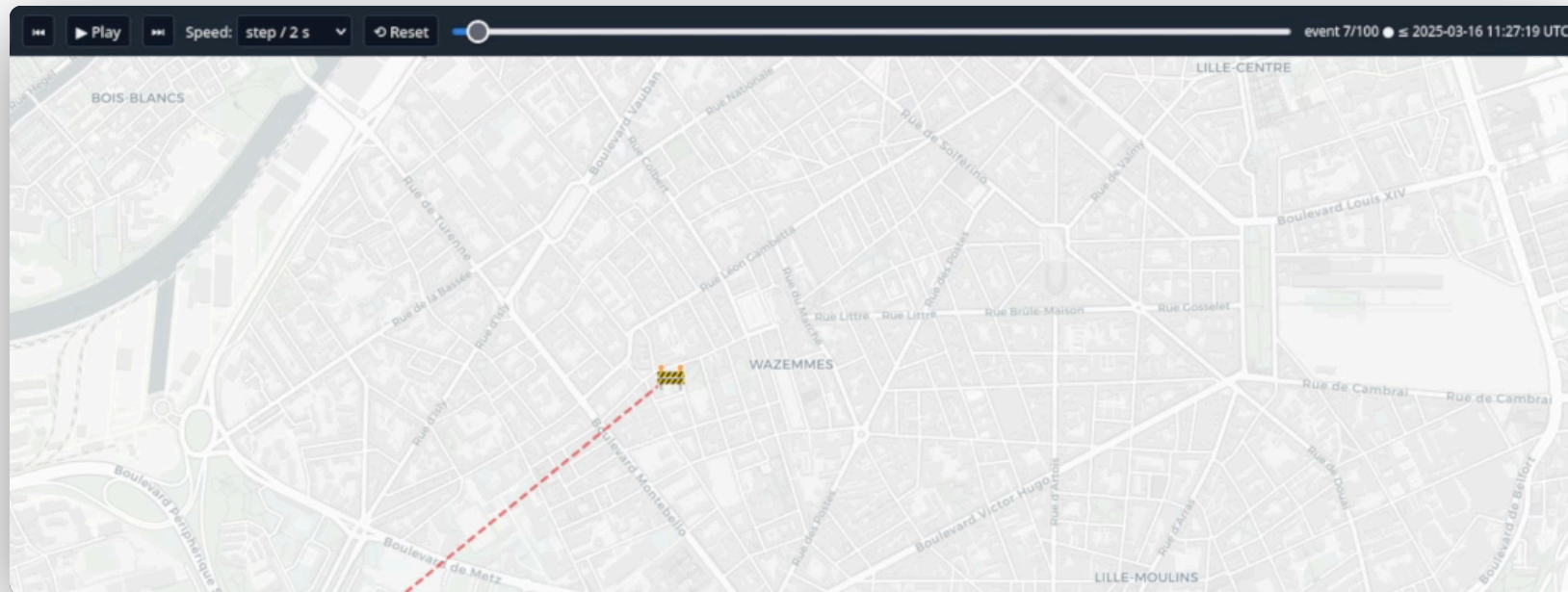
Un utilisateur... et son lieu de vacances



Détail amusant n°1



Des incidents signalés **d'un bout à l'autre de la France...**
en **quelques minutes.**



Détail amusant n°2



Très peu d'utilisateurs uniques.

event 100/100 ● ≤ 2026-03-08 17:50:44 UTC

 100 / 100 reports · 62 / 62 unique users (62 checked)



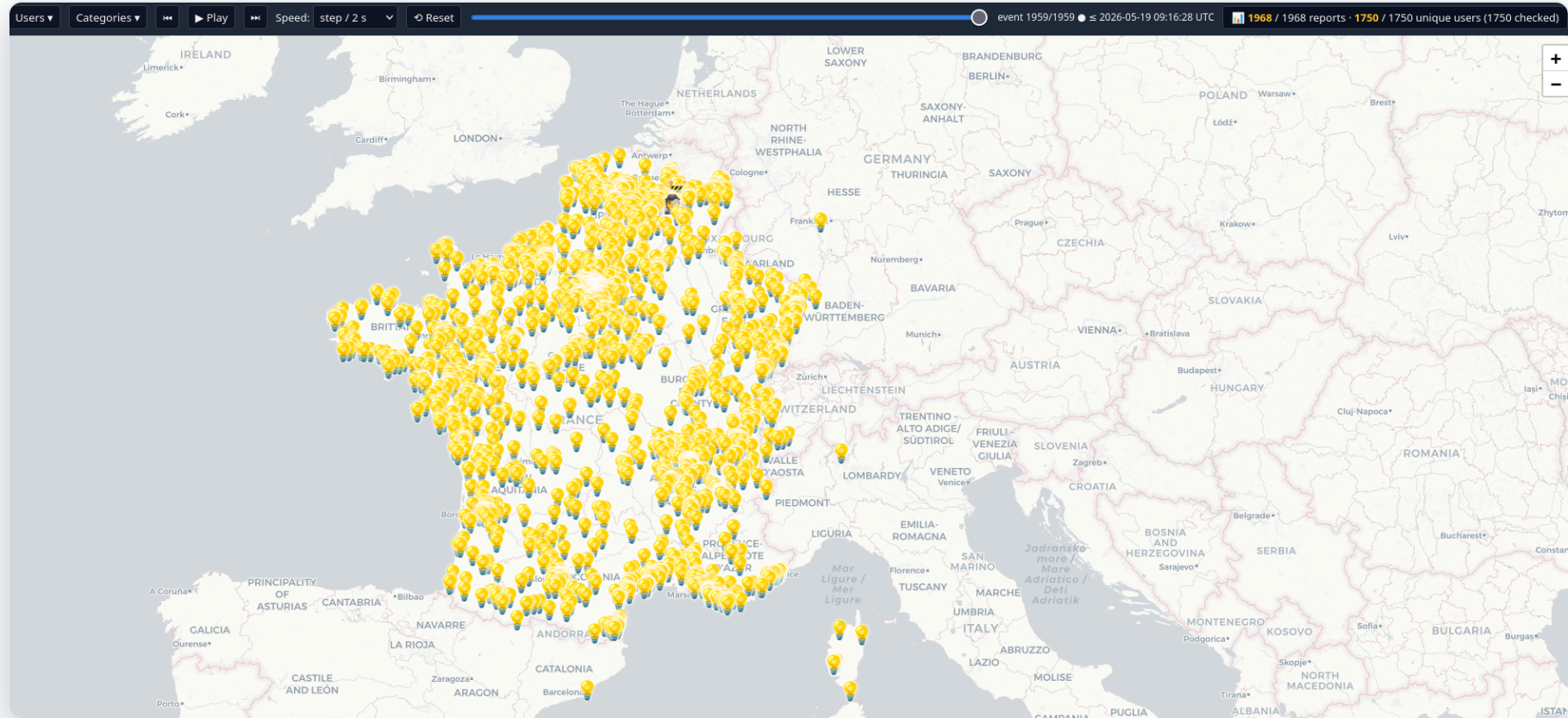
Quelques mois plus tard...



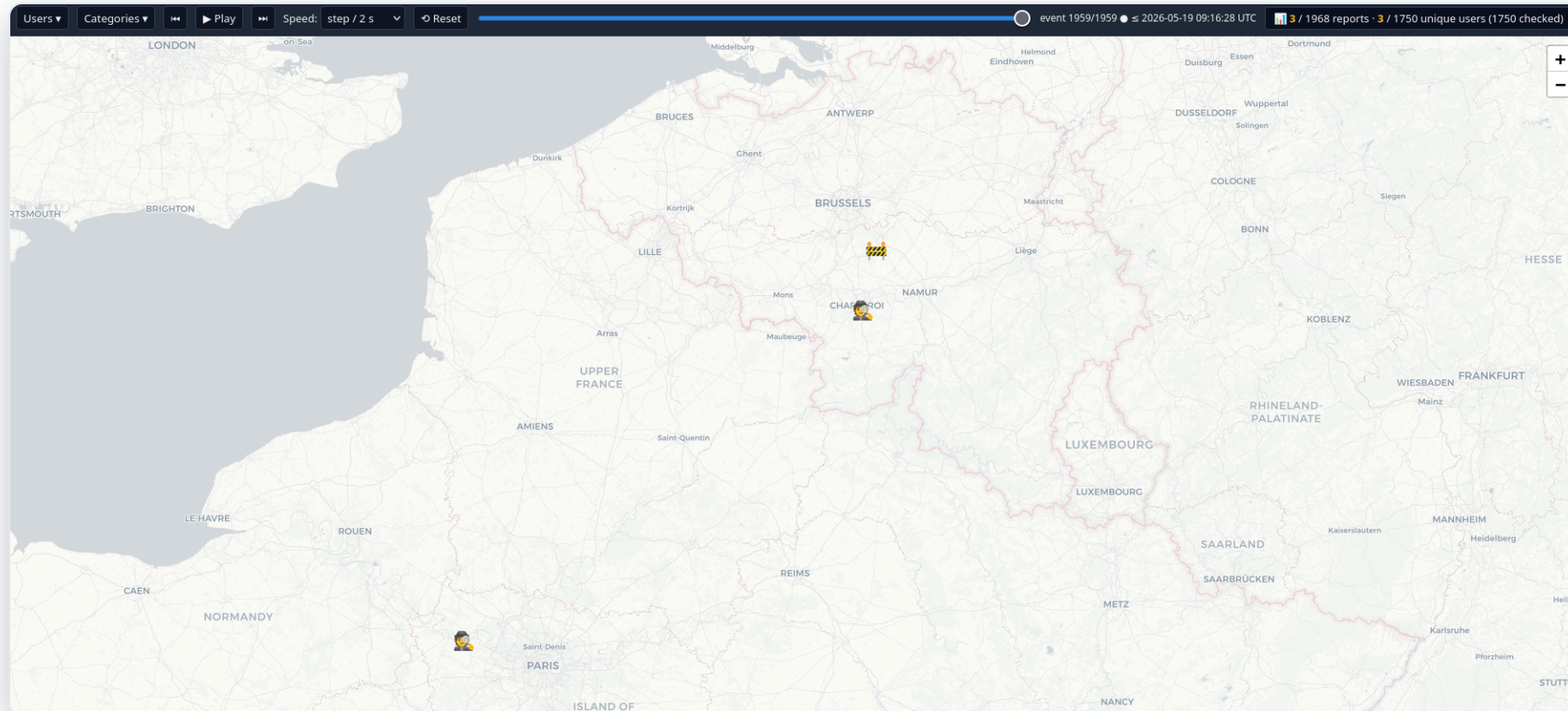
On revient voir si c'est corrigé.

- ▶ `reports` => **n'est plus accessible** directement ✓
- ▶ mais la fonction `getReport` => **redonne les mêmes données** ✗
- ▶ `events` => toujours là

La deuxième carte



La deuxième carte



Le partage de position en direct.

N° 117

Démarrer un partage : startSharing



```
async function startSharing(isFromTrustedCircle) {  
  const payload = { ...currentLocation, userId: auth.currentUser.uid, route: null };  
  const ref = await push(realtimeRef, payload);  
  setState({ sharedRef: ref, isFromTrustedCircle });  
  return ref.key; // = SHAREDREF  
}
```

Cette clé renvoyée, c'est le **SHAREDREF** l'identifiant du partage, celui qu'on envoie à ses proches.

Mettre à jour : setLocation



```
const payload = {  
  ...location,           // coords, timestamp, mocked  
  userId:      auth.currentUser.uid,  
  batteryLevel: batteryLevel,  
  coords:      { latitude, longitude },  
  route:       route,     // polyline du trajet  
  destination: store.getState().history[0].address  
};  
await update(sharedRef, payload);    // PATCH /<SHAREDREF>.json
```


Tout passe en REST (et en SSE)



```
curl -s -X PATCH "$RTDB/$SHAREDREF.json?auth=$ID" -d '{...}' # modifier
curl -s -X DELETE "$RTDB/$SHAREDREF.json?auth=$ID" # supprimer
curl -N "$RTDB/$SHAREDREF.json?auth=$ID" -H "Accept: text/event-stream" # suivre en direct
```

Preuve : l'UID fuit



Deux comptes anonymes. **B** lit le partage de **A**.

```
{ "coords": { "latitude": 48.857, "longitude": 2.353 },  
  "destination": "12 rue Exemple, 75001 Paris",  
  "batteryLevel": 87,  
  "userId": "ZsCUAo0z6eNn0XXo9Di1F9WW58y1" }  <= = UID de A, bit pour bit
```

Ce que ça implique



- ▶ Le tiers n'est pas spectateur : il peut **modifier** ou **supprimer** votre position
- ▶ Avec votre **UID**, il peut suivre **tous vos signalements**
- ▶ Destination + batterie en prime, **tout est falsifiable**

*Dé-anonymisation **horaire** et **géographique** d'utilisateurs supposément anonymes.*

Que ce soit par pseudonymisation cassée... ou en connaissant **exactement** la personne.



Bonus théorique : le SHAREDREF



Firebase : identifiant de **72 bits**, « aléatoire **et** incrémental ».

Si c'est incrémental... l'espace de recherche se réduit à l'intervalle entre deux IDs générés.

Deux clés d'API **en clair** dans l'application :

▶  **Mapbox**

▶  **HERE**

Ce qui devrait, soyons d'accord, être fait côté serveur.

Derrière l'humour.

N° 117

Annoncé	Mesuré
émulateur GPS / anti-spoof	/
anti-VPN	/
IA comportementale	<code>if distance > 50m (côté client)</code>

473

communes utilisent ce service

La sécurité reposait sur des **déclarations**, pas sur des **mécanismes**.

- ▶ « inspirable » n'est pas une mesure de sécurité
- ▶ « pas d'utilisateurs » n'est pas un correctif
- ▶ « hors modèle de menace » n'est pas une réponse

Merci.

Questions ?

Agent 117 / mission terminée

Remerciements (sincères) aux coordinateurs, à Photon Group, Olago et Nodus (ma graphiste)

L'application : un seul bouton. Allumer.

